

Отчёт по лабораторной работе №8

Дисциплина: Архитектура компьютеров

Мухина Ксения Николаевна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Задания для самостоятельной работы	11
4	Выводы	13
	Список литературы	14

Список иллюстраций

2.1	Создание каталога и файла для работы.	6
2.2	Программа lab8-1.	6
2.3	Изменение программы lab8-1.	7
2.4	Изменённая программа lab8-1.	7
2.5	Изменение программы lab8-1.	8
2.6	Изменённая программа lab8-1.	8
2.7	Программа lab8-2.	9
2.8	Программа lab8-3.	9
2.9	Изменение программы lab8-3.	10
2.10	Изменённая программа lab8-3.	10
3.1	Программа task.	12

Список таблиц

1 Цель работы

Приобрести навыки написания программ с использованием циклов и с обработкой аргументов командной строки.

2 Выполнение лабораторной работы

Далее описываемая работа была выполнена на виртуальной машине Oracle VirtualBox с ОС Linux Ubuntu.

1. Реализация циклов в NASM

Перед началом работы создадим каталог lab08 и файл lab8-1.asm.

```
knmuhina@virtualbox:~$ mkdir ~/work/arch-pc/lab08
knmuhina@virtualbox:~$ cd ~/work/arch-pc/lab08
knmuhina@virtualbox:~/work/arch-pc/lab08$ touch lab8-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab08$
```

Рисунок 2.1: Создание каталога и файла для работы.

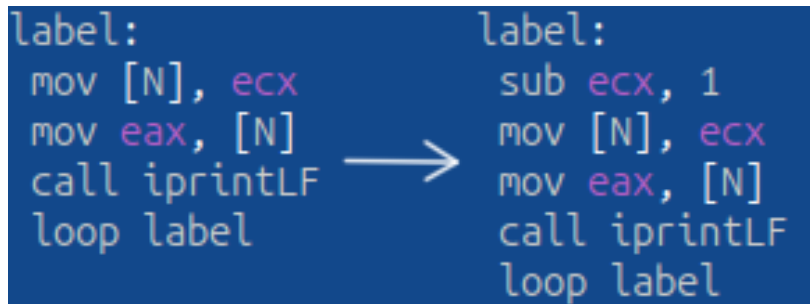
Рассмотрим первый пример использования инструкции loop. Введём в файл текст программы из листинга, представленного в лабораторной работе, и создадим исполняемый файл.

```
knmuhina@virtualbox:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
knmuhina@virtualbox:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 15
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
knmuhina@virtualbox:~/work/arch-pc/lab08$
```

Рисунок 2.2: Программа lab8-1.

Инструкция `loop` использует регистр `ecx` в качестве счётчика и на каждом шаге уменьшает значение на 1.

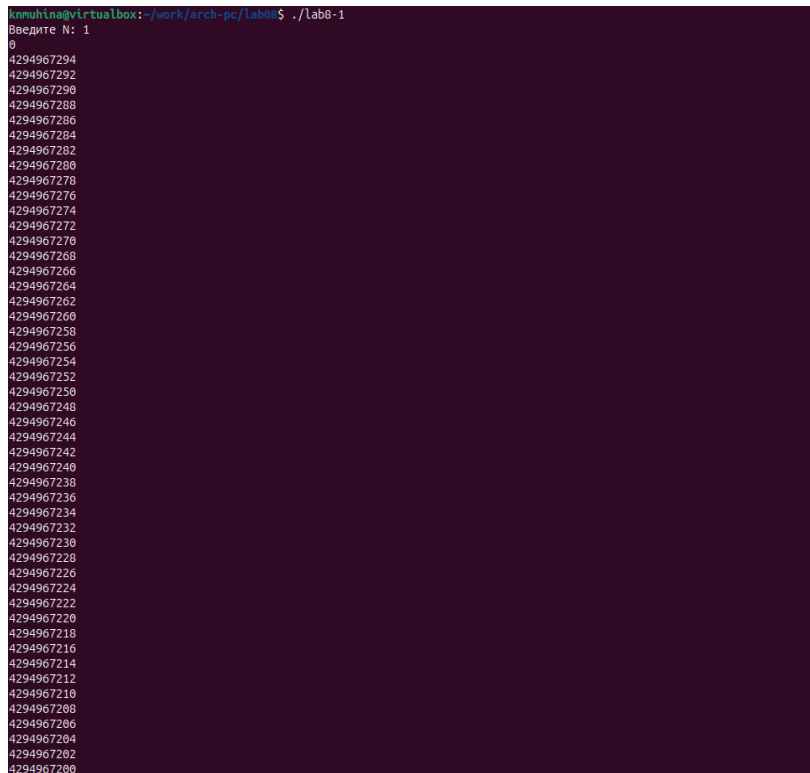
Изменения в регистре `ecx` внутри цикла могут привести к некорректной работе программы. Изменим текст программы следующим образом:



```
label:                                label:
  mov [N], ecx                        sub ecx, 1
  mov eax, [N]                        mov [N], ecx
  call iprintLF                       mov eax, [N]
  loop label                           call iprintLF
                                      loop label
```

Рисунок 2.3: Изменение программы lab8-1.

Создадим исполняемый файл и запустим его.



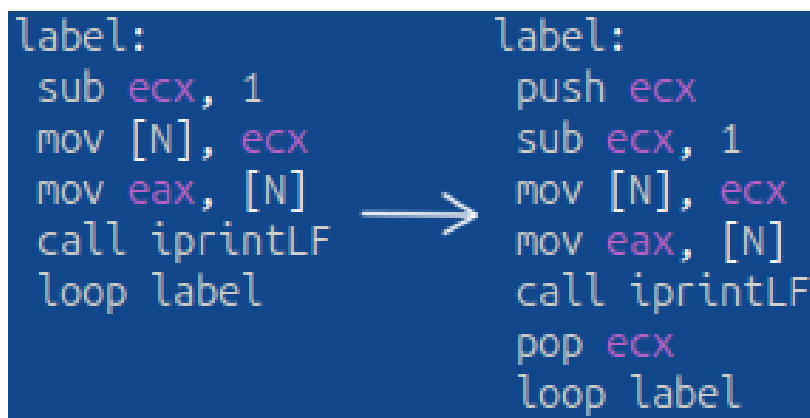
```
knnuhina@virtualbox:~/work/arch-pc/lab8$ ./lab8-1
Введите N: 1
0
4294967294
4294967292
4294967290
4294967288
4294967286
4294967284
4294967282
4294967280
4294967278
4294967276
4294967274
4294967272
4294967270
4294967268
4294967266
4294967264
4294967262
4294967260
4294967258
4294967256
4294967254
4294967252
4294967250
4294967248
4294967246
4294967244
4294967242
4294967240
4294967238
4294967236
4294967234
4294967232
4294967230
4294967228
4294967226
4294967224
4294967222
4294967220
4294967218
4294967216
4294967214
4294967212
4294967210
4294967208
4294967206
4294967204
4294967202
4294967200
```

Рисунок 2.4: Изменённая программа lab8-1.

В ходе работы программы после введения числа 1 программа предсказуемо выдаёт 0, затем при запуске нового цикла значение регистра становится меньше нуля, вследствие чего в качестве следующего значения он берёт максимально возможное положительное число, и начинает отсчёт с него.

При вводе чётного значения программа будет пропускать нечётные шаги, следовательно, в данной программе число проходов цикла никогда не будет соответствовать заданному значению N.

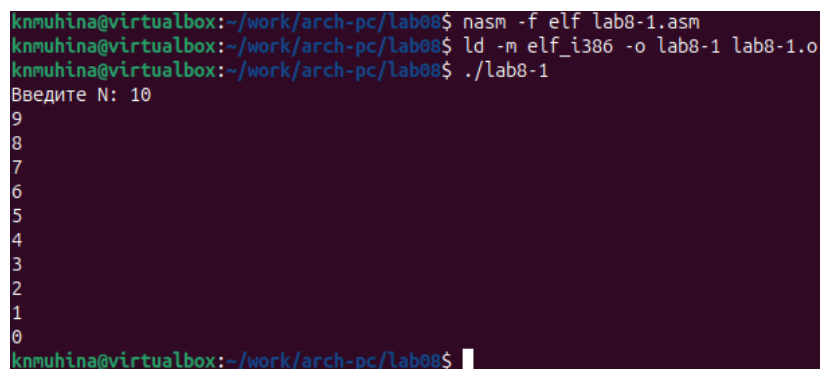
Для корректной работы программы такого вида можно использовать стек. Снова изменим текст программы:



```
label:                                label:
sub ecx, 1                            push ecx
mov [N], ecx                          sub ecx, 1
mov eax, [N]                          mov [N], ecx
call iprintLF                        mov eax, [N]
loop label                           call iprintLF
                                    pop ecx
                                    loop label
```

Рисунок 2.5: Изменение программы lab8-1.

Создадим исполняемый файл и запустим его.



```
knmuhina@virtualbox:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
knmuhina@virtualbox:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 10
9
8
7
6
5
4
3
2
1
0
knmuhina@virtualbox:~/work/arch-pc/lab08$
```

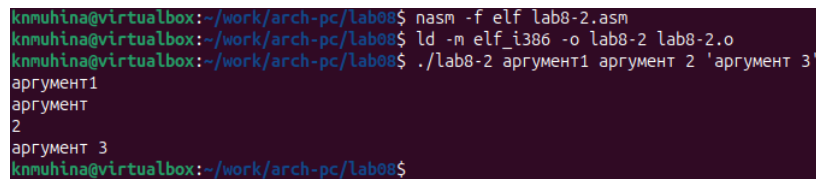
Рисунок 2.6: Изменённая программа lab8-1.

Теперь число проходов цикла соответствует заданному N, так как стек поз-

воляет сохранить значение счётчика цикла.

2. Обработка аргументов командной строки

В качестве примера обработки аргументов командной строки рассмотрим программу, выводящую на экран заданные аргументы. Создадим файл lab8-2.asm, введём в файл текст программы из листинга, представленного в лабораторной работе, и создадим исполняемый файл.

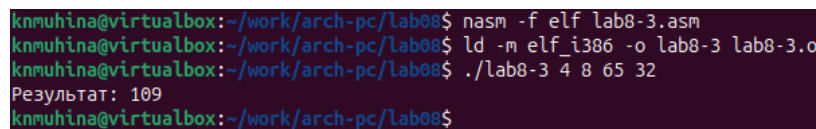


```
knmuhina@virtualbox:~/work/arch-pc/lab08$ nasm -f elf lab8-2.asm
knmuhina@virtualbox:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-2 lab8-2.o
knmuhina@virtualbox:~/work/arch-pc/lab08$ ./lab8-2 аргумент1 аргумент 2 'аргумент 3'
аргумент1
аргумент
2
аргумент 3
knmuhina@virtualbox:~/work/arch-pc/lab08$
```

Рисунок 2.7: Программа lab8-2.

Программа обработала 4 аргумента - „аргумент1“, „аргумент“, „2“ и „аргумент 3“.

Рассмотрим другой пример: программу, которая выводит сумму чисел, которые задаются как аргументы командной строки. Создадим файл lab8-3.asm, введём в файл текст программы из листинга, представленного в лабораторной работе, и создадим исполняемый файл.



```
knmuhina@virtualbox:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
knmuhina@virtualbox:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
knmuhina@virtualbox:~/work/arch-pc/lab08$ ./lab8-3 4 8 65 32
Результат: 109
knmuhina@virtualbox:~/work/arch-pc/lab08$
```

Рисунок 2.8: Программа lab8-3.

Программа работает корректно.

Изменим текст программы так, чтобы программа выводила произведение аргументов.

<pre> _start: pop ecx pop edx sub ecx, 1 mov esi, 0 next: cmp ecx, 0h jz _end pop eax call atoi add esi, eax loop next </pre>	→	<pre> _start: pop ecx pop edx sub ecx, 1 mov esi, 1 next: cmp ecx, 0h jz _end pop eax call atoi mul esi mov esi, eax loop next </pre>
--	---	--

Рисунок 2.9: Изменение программы lab8-3.

Создадим исполняемый файл и запустим его.

```

knmuhina@virtualbox:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
knmuhina@virtualbox:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
knmuhina@virtualbox:~/work/arch-pc/lab08$ ./lab8-3 4 8 5
Результат: 160
knmuhina@virtualbox:~/work/arch-pc/lab08$

```

Рисунок 2.10: Изменённая программа lab8-3.

Программа работает корректно.

3 Задания для самостоятельной работы

Создадим файл task.asm и напишем программу вычисления суммы значений заданной функции. Вид функции определён вариантом, полученным при выполнении лабораторной работы №6.

Листинг 1. Программа task.asm

```
%include 'in_out.asm'

SECTION .data
msg db 'Функция:  $f(x) = 15x - 9$ ', 0h
msgR db 'Результат: ', 0h

SECTION .text
global _start
_start:
    pop ecx
    pop edx
    sub ecx, 1
    mov esi, 0

next:
```

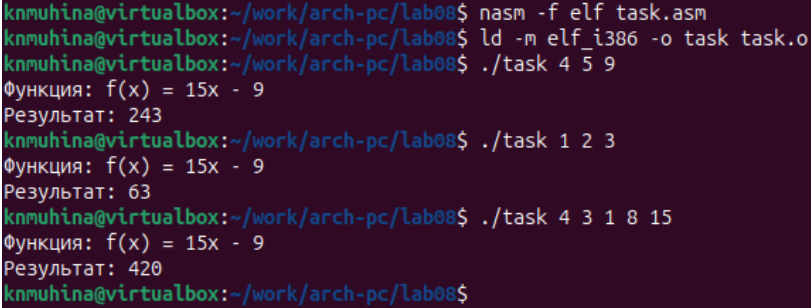
```

cmp ecx, 0
jz _end
pop eax
call atoi
mov ebx, 15
mul ebx
sub eax, 9
add esi, eax
loop next

_end:
mov eax, msg
call sprintfLF
mov eax, msgR
call sprintf
mov eax, esi
call iprintLF
call quit

```

Создадим исполняемый файл и проверим работу программы для различных наборов аргументов.



```

knmuuhina@virtualbox:~/work/arch-pc/lab08$ nasm -f elf task.asm
knmuuhina@virtualbox:~/work/arch-pc/lab08$ ld -m elf_i386 -o task task.o
knmuuhina@virtualbox:~/work/arch-pc/lab08$ ./task 4 5 9
Функция: f(x) = 15x - 9
Результат: 243
knmuuhina@virtualbox:~/work/arch-pc/lab08$ ./task 1 2 3
Функция: f(x) = 15x - 9
Результат: 63
knmuuhina@virtualbox:~/work/arch-pc/lab08$ ./task 4 3 1 8 15
Функция: f(x) = 15x - 9
Результат: 420
knmuuhina@virtualbox:~/work/arch-pc/lab08$

```

Рисунок 3.1: Программа task.

Программа работает корректно.

4 Выводы

В ходе выполнения лабораторной работы мы приобрели навыки написания программ с использованием циклов и с обработкой аргументов командной строки.

Список литературы

1. Файл «Лабораторная работа №8. Программирование цикла. Обработка аргументов командной строки.pdf»